Heaven's Light Is Our Guide

Rajshahi University Of Engineering & Technology

Department Of Electrical & Computer Engineering

Lab Report-01

Course Title : Digital Techniques Sessional

Course Code : ECE-2112

Date Of Submission : 01.08.26

Submitted By	Submitted To
Nishit Ray Roll : 2410044	Mst. Mazeda Noor Tasnim Lecturer, Department of ECE RUET

Circuit Design

Task 1: OR Gate Implementation using NOR Gates

Objectives

- To understand the working principle of a NOR gate.
- To design and simulate an OR gate using only NOR gates.
- To verify the constructed circuit against the standard OR gate truth table.

Apparatus / Tools Used

Digital logic circuit simulation software, NOR gate blocks, input switches, output indicator .

Introduction

An OR gate outputs HIGH unless both inputs are LOW, and because NOR gates are universal, any logic function can be built using them alone; thus, this experiment constructs an OR gate solely from NOR logic and verifies its operation against standard OR truth table values.

Theory

A NOR gate outputs the complement of what an OR gate would give for the same two inputs, so its output is defined as

$$Y = (A + B)'$$

A	B	$Y = (A + B)'$
0	0	1
0	1	0
1	0	0
1	1	0

Table 1: Truth Table of NOR Gate

If a signal is passed through a NOR gate twice, the double inversion cancels out and the original OR expression is recovered:

$$A + B = ((A + B)')'$$

To create the circuit, inputs A and B were first routed into a single NOR gate to generate the inverted expression (A + B)'. This output was then fed into both tied inputs of a second NOR gate, forcing it to act as an inverter and yielding the final restored OR output, A + B.

Circuit Design

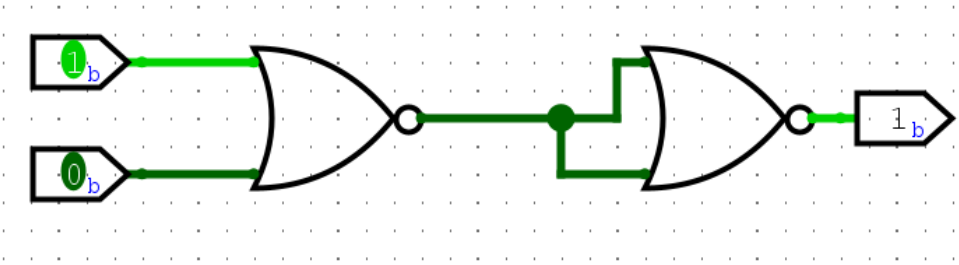

Figure 1: OR using NOR

Result and Discussion

The circuit was tested for all four possible combinations of A and B, and the output obtained in each case was compared against the expected OR gate output.

A	B	Expected Output	Obtained Output
0	0	0	0
0	1	1	1
1	0	1	1
1	1	1	1

Table 2: Testing of OR Gate using NOR Gates

In every case, the obtained output matched the expected result, confirming that the circuit correctly performs the OR operation.

Conclusion

The experiment demonstrated that an OR gate can be fully built using only NOR gates, as chaining two of them together—with the second acting as an inverter—reproduces the OR logic perfectly. This confirms the universal nature of the NOR gate.

Task 2: OR Gate Implementation using NAND Gates

Objectives

- To understand the working principle of a NAND gate.
- To design and simulate an OR gate using only NAND gates.
- To verify the constructed circuit against the standard OR gate truth table.

Apparatus / Tools Used

Digital logic circuit simulation software, NAND gate blocks, input switches, output indicator .

Circuit Design

Introduction

Like the NOR gate, the NAND gate is a universal gate because you can build any logic function—including an OR gate—using only NAND gates. This experiment focuses on constructing an OR gate exclusively with NAND gates, using De Morgan's theorem to switch between the two logic forms.

Theory

A NAND gate outputs the complement of what an AND gate would give for the same two inputs:

$$Y = (A \cdot B)'$$

A	B	$Y = (A \cdot B)'$
0	0	1
0	1	1
1	0	1
1	1	0

Table 3: Truth Table of NAND Gate

By De Morgan's theorem, the OR operation can be rewritten entirely in terms of complemented AND:

$$A + B = (A' \cdot B')'$$

The expression $(A \cdot B)' = A + B$ shows that combining A' and B' through a NAND gate yields A + B, proving that an OR gate can be made entirely from NAND gates. In this setup, two NAND gates with their inputs tied together act as basic inverters to generate A' and B' from inputs A and B. Feeding these two inverted signals into a third NAND gate performs the operation $(A' \cdot B')'$, which De Morgan's theorem confirms is identical to A + B, producing the final OR logic.

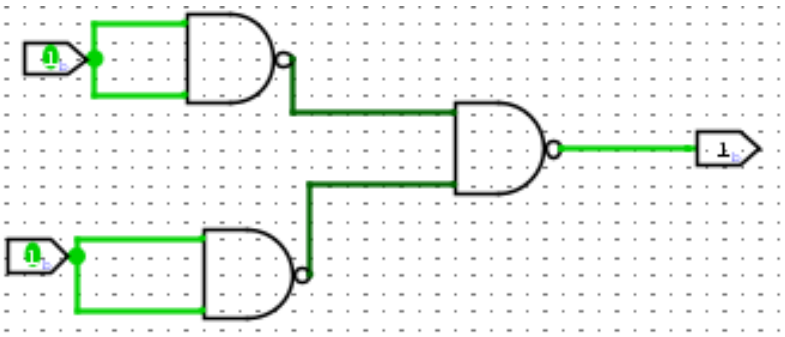

Figure 2: OR using NAND

Circuit Design

Result and Discussion

The circuit was tested for all four possible combinations of A and B, and the output obtained in each case was compared against the expected OR gate output.

A	B	Expected Output	Obtained Output
0	0	0	0
0	1	1	1
1	0	1	1
1	1	1	1

Table 4: Testing of OR Gate using NAND Gates

In every case, the obtained output matched the expected result, confirming that the circuit correctly performs the OR operation.

Conclusion

This experiment showed that an OR gate can be built entirely out of NAND gates by using two NAND gates as inverters to create A' and B', then feeding both into a third NAND gate. By applying De Morgan's theorem, this proves that the NAND gate is a universal component capable of performing the OR function.

Task 3: AND Gate Implementation using NOR Gates

Objectives

- To understand the working principle of a NOR gate.
- To design and simulate an AND gate using only NOR gates.
- To verify the constructed circuit against the standard AND gate truth table.

Apparatus / Tools Used

Digital logic circuit simulation software, NOR gate blocks, input switches, output indicator (LED).

Introduction

Having already built an OR gate with NOR gates, this experiment takes the concept a step further by constructing an AND gate entirely from NOR gates. Because the NOR gate is universal, we can achieve this using De Morgan's theorem, which connects AND and OR operations through complementation.

Theory

An AND gate produces a HIGH output only when both of its inputs are HIGH:

$$Y = A \cdot B$$

Circuit Design

A	B	$Y = A \cdot B$
0	0	0
0	1	0
1	0	0
1	1	1

Table 5: Truth Table of AND Gate

By De Morgan's theorem, the AND operation can be expressed entirely through complemented OR:

$$A \cdot B = (A' + B')'$$

This means that if A' and B' are produced first, and then combined through a NOR operation, the result is exactly $A \cdot B$. Since tying both inputs of a NOR gate together makes it behave as a simple inverter, this whole expression can be built using NOR gates only.

Two NOR gates were first set up as inverters – one for input A and one for input B – by connecting each gate's own input to both of its terminals, producing A' and B' respectively. These two inverted signals were then passed into a third NOR gate. Since this third gate receives A' and B', its output corresponds to $(A' + B')'$, which by De Morgan's theorem is equivalent to $A \cdot B$, giving the final AND result.

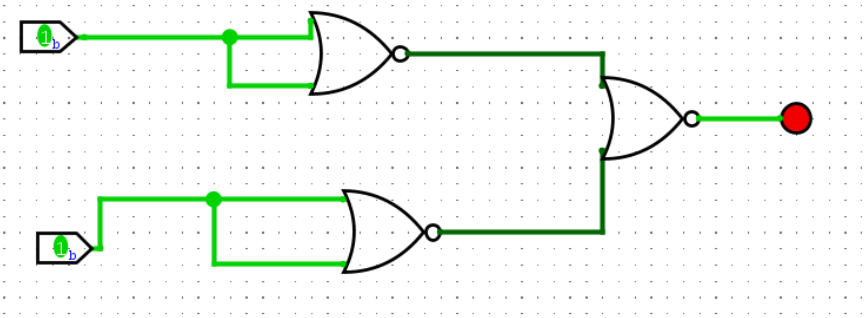

Figure 3: AND using NOR

Result and Discussion

The circuit was tested for all four possible combinations of A and B, and the output obtained in each case was compared against the expected AND gate output.

A	B	Expected Output	Obtained Output
0	0	0	0
0	1	0	0
1	0	0	0
1	1	1	1

Circuit Design

Table 6: Testing of AND Gate using NOR Gates

In every case, the obtained output matched the expected result, confirming that the circuit correctly performs the AND operation.

Conclusion

This experiment demonstrated that an AND gate can be fully realized using only NOR gates, by first generating A' and B' through two NOR gates wired as inverters, and then combining them through a third NOR gate. This confirms, by application of De Morgan's theorem, that the NOR gate is a universal gate capable of producing the AND function.

Task 4: AND Gate Implementation using NAND Gates

Objectives

- To understand the working principle of a NAND gate.
- To design and simulate an AND gate using only NAND gates.
- To verify the constructed circuit against the standard AND gate truth table.

Apparatus / Tools Used

Digital logic circuit simulation software, NAND gate blocks, input switches, output indicator (LED).

Introduction

Since a NAND gate is essentially an AND gate followed by an inverter, recovering a pure AND gate from NAND gates is a matter of cancelling out that inversion. This experiment builds an AND gate using two NAND gates connected in series and verifies its output against the standard AND truth table.

Theory

A NAND gate produces the complement of what an AND gate would output for the same two inputs:

$$Y = (A \cdot B)'$$

A	B	$Y = (A \cdot B)'$
0	0	1
0	1	1
1	0	1
1	1	0

Table 7: Truth Table of NAND Gate

Circuit Design

Passing a signal through a NAND gate twice undoes the inversion and recovers the original AND expression:

$$A \cdot B = ((A \cdot B)')'$$

This means the first NAND gate can be used to generate $(A \cdot B)'$ from inputs A and B, and a second NAND gate, wired so both of its terminals receive that same signal, acts purely as an inverter and restores the AND result.

Circuit Design

Inputs A and B were first applied to a NAND gate, producing $(A \cdot B)'$ at its output. This signal was then fed into both inputs of a second NAND gate simultaneously, causing that

Circuit Design

gate to function as a simple inverter. The output taken from this second gate is therefore the final AND result, $A \cdot B$.

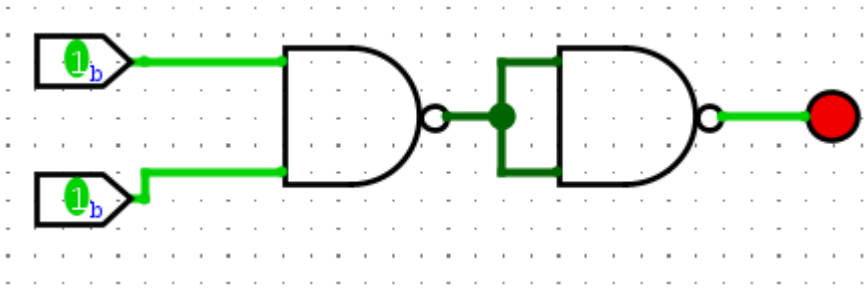

Figure 4: AND using NAND

Result and Discussion

The circuit was tested for all four possible combinations of A and B, and the output obtained in each case was compared against the expected AND gate output.

A	B	Expected Output	Obtained Output
0	0	0	0
0	1	0	0
1	0	0	0
1	1	1	1

Table 8: Testing of AND Gate using NAND Gates

In every case, the obtained output matched the expected result, confirming that the circuit correctly performs the AND operation.

Conclusion

This experiment demonstrated that an AND gate can be fully realized using only NAND gates, since two NAND gates in series – the second acting as an inverter – reproduce the AND function exactly. This confirms the universal nature of the NAND gate.

Task 5: NOT Gate Implementation using NOR Gates

Objectives

- To understand the working principle of a NOR gate.
- To design and simulate a NOT gate using only a NOR gate.
- To verify the constructed circuit against the standard NOT gate truth table.

Circuit Design

Apparatus / Tools Used

Digital logic circuit simulation software, NOR gate block, input switch, output indicator (LED).

Introduction

Unlike the previous experiments that handled two separate inputs, a NOT gate operates on just one input and simply flips its state. This experiment demonstrates that when both inputs of a NOR gate are tied to the same signal, it mimics this exact behavior—functioning directly as a standard inverter.

Theory

A NOT gate outputs the complement of its single input:

A	$Y = A'$
0	1
1	0

Table 9: Truth Table of NOT Gate

If both inputs of a NOR gate are connected to the same signal A, its output reduces to

$$A' = A \text{ NOR } A$$

which is precisely the NOT operation. This works because the NOR expression $(A+A)'$ simplifies to A', since $A + A = A$.

Circuit Design

A single input, A, was connected to both terminals of a NOR gate simultaneously. With both inputs identical, the gate's OR stage produces A internally, which is then inverted by the NOR gate's built-in complement stage, so the final output observed is A'.

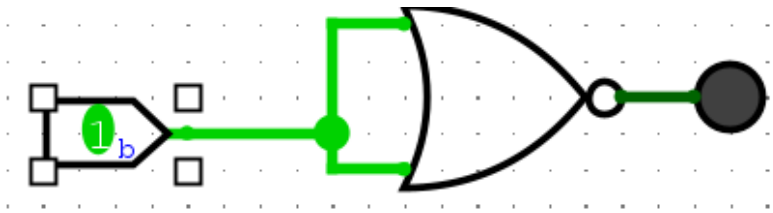

Figure 5: NOT using NOR

Result and Discussion

The circuit was tested for both possible values of A, and the output obtained in each case was compared against the expected NOT gate output.

Circuit Design

A	Expected Output	Obtained Output
0	1	1
1	0	0

Table 10: Testing of NOT Gate using NOR Gate

In both cases, the obtained output matched the expected result, confirming that the circuit correctly performs the NOT operation.

Conclusion

This experiment showed that a NOR gate turns into a NOT gate simply by tying both of its inputs to the same signal. This further confirms the universal switching capability of the NOR gate, proving it can perform even the most basic logic operation.

Task 6: NOT Gate Implementation using NAND Gates

Objectives

- To understand the working principle of a NAND gate.
- To design and simulate a NOT gate using only a NAND gate.
- To verify the constructed circuit against the standard NOT gate truth table.

Apparatus / Tools Used

Digital logic circuit simulation software, NAND gate block, input switch, output indicator (LED).

Introduction

This experiment rounds out our single-gate inverter setups by demonstrating that a NAND gate—just like a NOR gate—can also function as a NOT gate. Tying both of its inputs to the same signal simplifies its usual two-input behavior into a straight-forward inversion.

Theory

A NOT gate outputs the complement of its single input:

	$Y = A'$
A	$Y = A'$
0	1
1	0

Table 11: Truth Table of NOT Gate

If both inputs of a NAND gate are connected to the same signal A, its output reduces to

Circuit Design

$$A' = A \text{ NAND } A$$

which is precisely the NOT operation. This follows because the NAND expression $(A \cdot A)'$ simplifies to A', since $A \cdot A = A$.

Circuit Design

A single input, A, was connected to both terminals of a NAND gate simultaneously. With both inputs identical, the gate's internal AND stage produces A, which is then inverted by the NAND gate's complement stage, so the final output observed is A'.

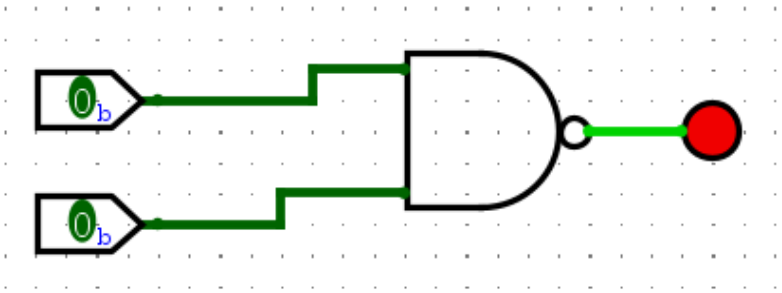

Figure 6: NOT using NAND

Result and Discussion

The circuit was tested for both possible values of A, and the output obtained in each case was compared against the expected NOT gate output.

A	Expected Output	Obtained Output
0	1	1
1	0	0

Table 12: Testing of NOT Gate using NAND Gate

In both cases, the obtained output matched the expected result, confirming that the circuit correctly performs the NOT operation.

Conclusion

This experiment demonstrated that a NAND gate can be reduced to a NOT gate simply by connecting both of its inputs to the same signal. Together with the previous experiment, this confirms that both the NOR and NAND gates are capable of realizing the NOT operation, reinforcing their status as universal gates.

Circuit Design

Task 7: Implementation of Full Adder and Testing

Objectives

- To understand the working principle of a Full Adder circuit.
- To construct a Full Adder using two Half Adders and an OR gate.
- To verify the constructed circuit against the standard Full Adder truth table for all input combinations.

Apparatus / Tools Used

Digital logic circuit simulation software, XOR gates, AND gates, OR gate, input switches, output indicators (LEDs).

Introduction

Moving beyond the earlier experiments focused on single logic gates, this experiment highlights a combinational circuit made of multiple gates working together. A Full Adder builds on the Half Adder by taking a third input—a carry-in—allowing multi-bit binary addition by adding numbers in sequence. Here, we construct a Full Adder using two Half Adder blocks and verify its Sum and Carry-out outputs across all eight possible input combinations.

Theory

A Full Adder takes three single-bit inputs, A, B, and a carry-in (C_{in}), and produces two outputs: the Sum and the carry-out (C_{out}). These are expressed as

$$Sum = A \oplus B \oplus C_{in}$$

$$C_{out} = AB + C_{in}(A \oplus B)$$

Looking at the Sum expression, we can see that it simply represents an XOR operation across three signals, which is easily built by chaining two Half Adders—the first combining A and B, and the second pairing that output with C_{in}. Meanwhile, the carry-out goes HIGH whenever either Half Adder generates a carry, meaning the two separate carry signals can just be combined through an OR gate to produce the final C_{out}.

A	B	C_{in}	Sum	C_{out}
0	0	0	0	0
0	0	1	1	0
0	1	0	1	0
0	1	1	0	1
1	0	0	1	0
1	0	1	0	1
1	1	0	0	1
1	1	1	1	1

Circuit Design

Table 13: Truth Table of Full Adder

Circuit Design

The first Half Adder was set up to take inputs A and B, generating a partial sum along with its own carry output. This partial sum was then fed into a second Half Adder alongside C_{in}, which produced the final Sum output directly. Finally, the carry outputs from both Half Adders were routed into an OR gate, with its output serving as the Full Adder's overall Carry-out, C_{out}.

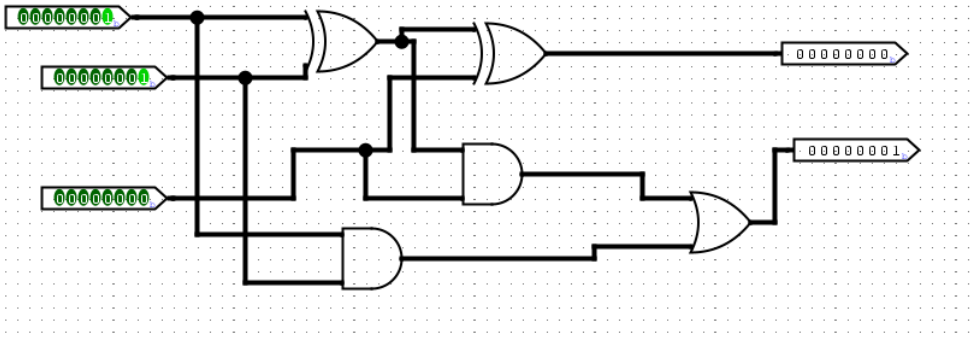

Figure 7: Full Adder circuit

Result and Discussion

Since a Full Adder takes three inputs, it was tested across all eight possible combinations of A, B, and C_{in}, recording both the Sum and Carry-out outputs against expected values. For every single combination, the measured Sum and Carry-out matched the theoretical results perfectly, confirming that the circuit accurately performs as a Full Adder.

A	B	C_{in}	Expected (Sum, C_{out})	Obtained (Sum, C_{out})
0	0	0	0, 0	0, 0
0	0	1	1, 0	1, 0
0	1	0	1, 0	1, 0
0	1	1	0, 1	0, 1
1	0	0	1, 0	1, 0
1	0	1	0, 1	0, 1
1	1	0	0, 1	0, 1
1	1	1	1, 1	1, 1

Table 14: Testing of Full Adder

Circuit Design

Conclusion

This experiment demonstrated that a Full Adder can be successfully built by cascading two Half Adders and combining their carry outputs through an OR gate. The verified results across all eight input combinations confirm correct three-bit binary addition, forming the basis for larger multi-bit adders.

Task 8: Implementation of Binary to BCD Converter

Objectives

- To understand the concept of Binary-Coded Decimal (BCD) representation.
- To design and simulate a 4-bit Binary to BCD Converter circuit.
- To verify the circuit's output against the expected BCD values for various binary inputs.

Apparatus / Tools Used

Digital logic circuit simulation software, Binary-to-BCD converter block, input switches, seven-segment BCD display.

Introduction

Digital circuits naturally operate in binary, but decimal is much easier for humans to read—like on a seven-segment display. This experiment implements a converter that takes a 4-bit binary input and translates it into Binary-Coded Decimal (BCD) format.

Theory

A 4-bit binary input can represent values 0 to 15. For values 0–9, the BCD output matches the binary input directly, since one decimal digit fits in 4 bits. For values 10 and above, the number is split across two BCD digit groups – one per decimal digit. For example,

$$1010_2 = 10_{10} \rightarrow 0001\ 0000_{BCD}$$

The complete mapping for all 16 possible inputs is shown below.

B3	B2	B1	B0	D4	D3	D2	D1
0	0	0	0	0	0	0	0
0	0	0	1	0	0	0	1
0	0	1	0	0	0	1	0
0	0	1	1	0	0	1	1
0	1	0	0	0	1	0	0
0	1	0	1	0	1	0	1
0	1	1	0	0	1	1	0
0	1	1	1	0	1	1	1
1	0	0	0	1	0	0	0
1	0	0	1	1	0	0	1

Circuit Design

1	0	1	0	1	0	1	0
1	0	1	1	1	0	1	1
1	1	0	0	1	1	0	0
1	1	0	1	1	1	0	1
1	1	1	0	1	1	1	0
1	1	1	1	1	1	1	1

Table 15: Truth Table of Binary to BCD Converter

Circuit Design

The four binary inputs (B_3 through B_0) were routed into the Binary-to-BCD Converter block, which determines if the value goes above 9 and divides it between two BCD digit groups when necessary. The generated BCD output was then hooked up to a seven-segment display to easily view the matching decimal digit or digits.

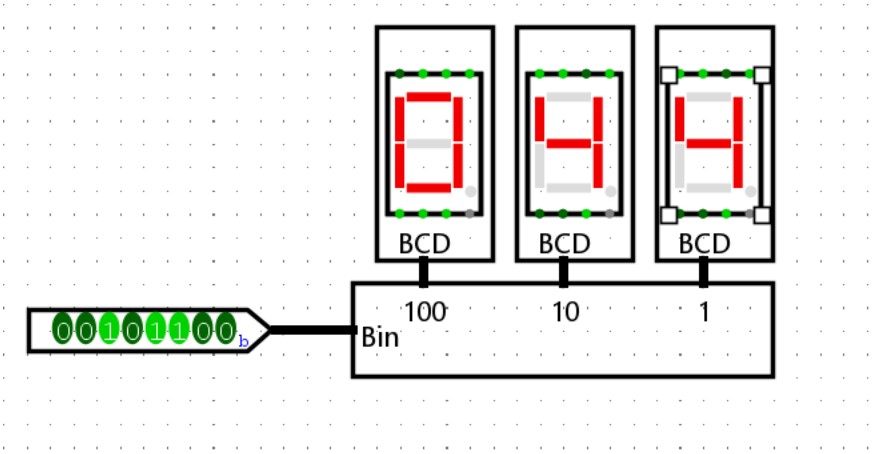

Figure 8: Binary to BCD

Result and Discussion

A range of binary inputs, including values both below and above 9, were applied to the converter, and the resulting BCD output was compared against the expected decimal equivalent in each case.

Binary Input	Decimal Equivalent	BCD Output
0000	0	0000
0001	1	0001
0101	5	0101
1001	9	1001
1010	10	0001 0000
1111	15	0001 0101

Circuit Design

Table 16: Testing of Binary to BCD Converter

The observed BCD outputs matched the expected decimal values across all tested inputs, including the cases where the value exceeded 9 and had to be split across two BCD digit groups.

Conclusion

This experiment demonstrated that a 4-bit binary number can be successfully converted into its Binary-Coded Decimal equivalent, with values over 9 properly split across two distinct digit groups. The verified results highlight the practical importance of these converters in driving decimal displays from binary digital systems.

References

[1] D. M. Harris and S. L. Harris, “Digital design and computer architecture, 2nd edition,” *Digital Design and Computer Architecture, 2nd Edition*, pp. 1–690, Jan. 2012, doi: 10.1016/C2011-0-04377-6.

[2] **Logisim-evolution** , Digital Logic Design Simulator, “Available: https://github.com/logisim-evolution/logisim-evolution